

YouTube Chat RAG

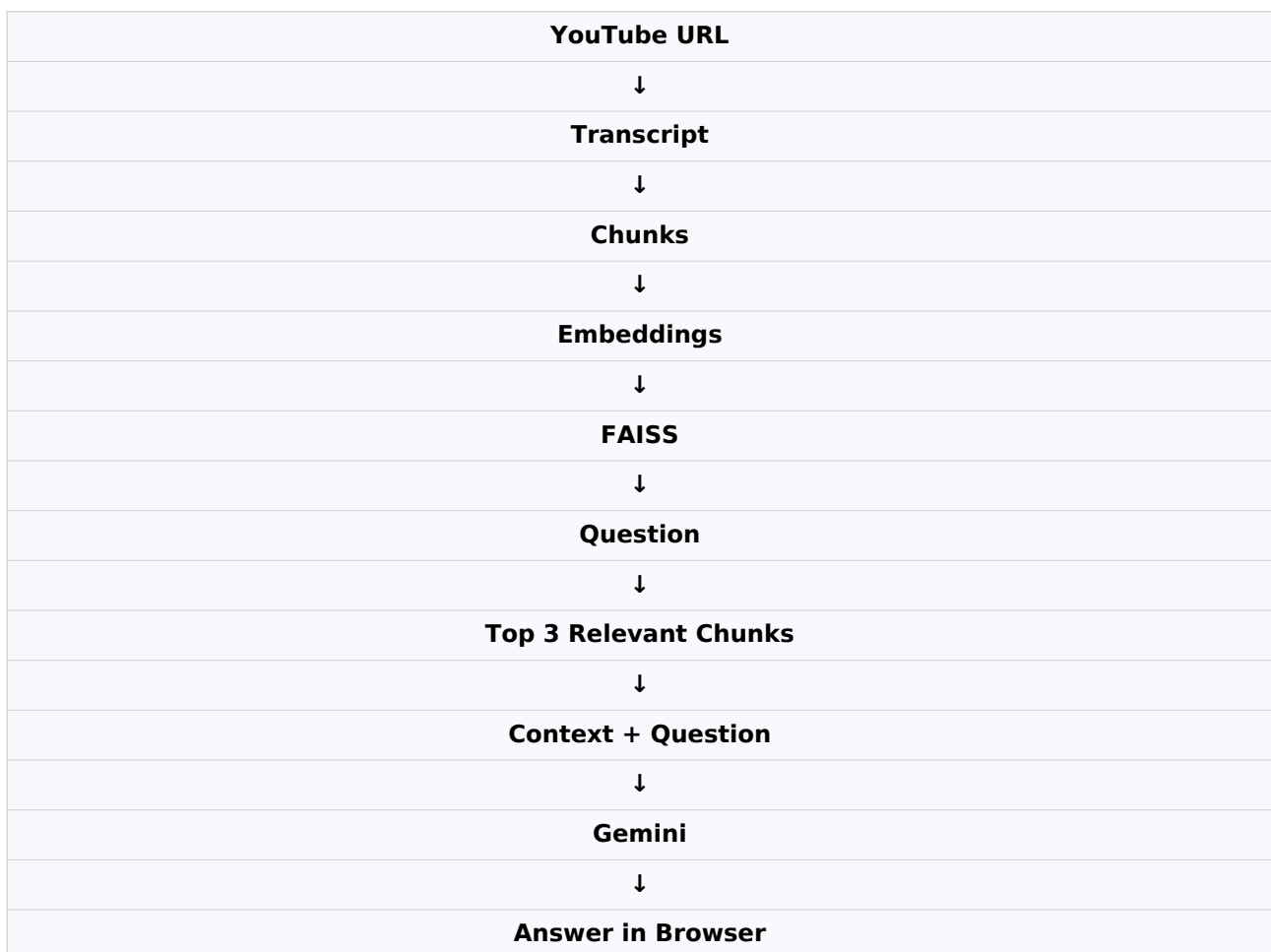
Project Reference & Learning Guide

Purpose: A simple permanent reference for understanding what the application does, why each component is used, and how the complete RAG + FastAPI flow works.

1. Project at a Glance

What is it? A local web application where a user pastes a YouTube URL and then asks questions about that video's transcript.

Problem solved: Instead of manually searching a long video, the user can ask a question and get an answer based on relevant transcript content.



2. Technologies

Technology	Role
Python	Runs the application and RAG pipeline.
FastAPI	Connects the browser to the Python logic.
Uvicorn	Runs the FastAPI application locally.
HTML	Provides the basic webpage and forms.
YouTube Transcript API	Fetches the available video transcript.

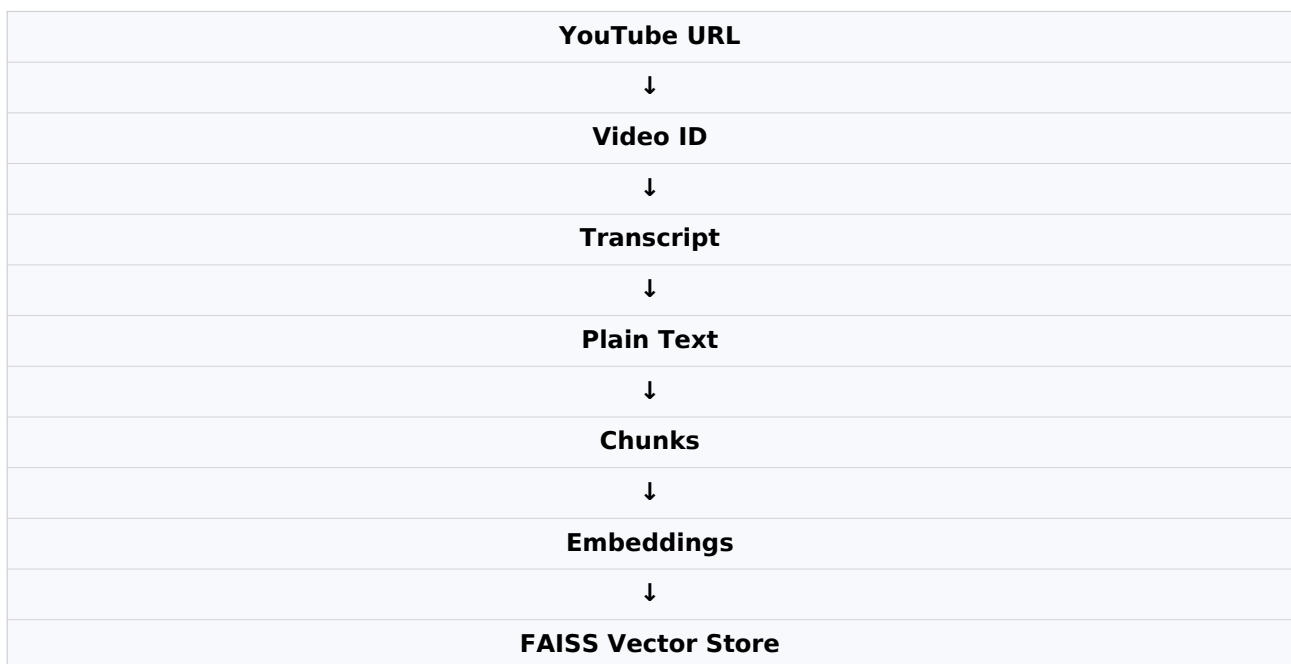
Technology	Role
LangChain Text Splitter	Splits transcript text into smaller chunks.
Gemini Embeddings	Converts text into numerical vectors.
FAISS	Indexes vectors and performs similarity search.
Gemini LLM	Generates the final answer.
python-dotenv	Loads the API key from .env.

3. RAG in Simple Words

RAG = Retrieval + Augmentation + Generation. First find the useful information, then add it to the question, then ask the LLM to generate the answer.

Stage	What happens	In this project
Retrieval	Find relevant information.	FAISS retrieves the top 3 transcript chunks.
Augmentation	Add retrieved information to the prompt.	Chunks + question become the context-aware prompt.
Generation	Generate the answer.	Gemini produces the final response.

4. Indexing Pipeline



Document Loading

```

api = YouTubeTranscriptApi()

transcript = api.fetch(
    video_id,
    languages=["en", "hi"]
)

```

Why? The transcript is the source information for the RAG system.

Text Extraction

```

text = " ".join(
    [snippet.text for snippet in transcript]
)

```

This takes the text from every transcript snippet and joins it into one normal string.

Chunking

```
text_splitter = RecursiveCharacterTextSplitter(  
    chunk_size=500,  
    chunk_overlap=50  
)  
  
chunks = text_splitter.split_text(text)
```

Why? A large transcript is easier to retrieve from when it is divided into smaller pieces. The current project uses 500 characters with 50 characters of overlap.

Embeddings

```
embeddings = GoogleGenerativeAIEmbeddings(  
    model="gemini-embedding-001"  
)
```

An embedding is a numerical representation of text meaning. It lets the system compare the meaning of a question with the meaning of transcript chunks.

FAISS

```
vector_store = FAISS.from_texts(  
    chunks,  
    embedding=embeddings  
)
```

FAISS creates the vector index using the supplied embedding model. Therefore, separately calling `embed_documents(chunks)` is unnecessary here and can cause duplicate embedding API requests.

5. Retrieval, Augmentation & Generation

Retrieval

```
results = vector_store.similarity_search(
    query,
    k=3
)
```

The user's question is used to find semantically similar transcript chunks. `k=3` means the top three results are selected.

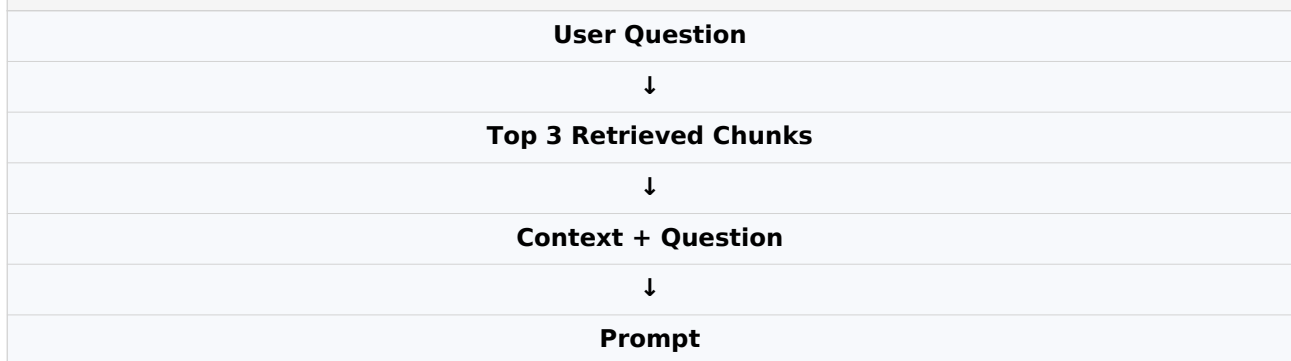
Augmentation

```
context = "\n\n".join(
    [result.page_content for result in results]
)

prompt = f"""
Answer the question using only the following context.

Context:
{context}

Question:
{query}
"""
```



Generation

```
llm = ChatGoogleGenerativeAI(
    model="gemini-3.6-flash"
)

response = llm.invoke(prompt)

answer = response.content[0]["text"]
```

Gemini receives the augmented prompt and generates the final answer. The last line extracts the answer text from the current structured response.

6. FastAPI — Project View

```
app = FastAPI()
```

Creates the FastAPI application.

```
@app.get("/", response_class=HTMLResponse)
def home():
    ...
```

GET / loads the basic homepage.

```
@app.post("/load-video", response_class=HTMLResponse)
def load_video(youtube_url: str = Form(...)):
    global vector_store
```

POST /load-video receives the YouTube URL from the HTML form. Form(...) tells FastAPI to read form data. global vector_store lets the function update the vector store variable created outside the function.

```
@app.post("/ask", response_class=HTMLResponse)
def ask_question(query: str = Form(...)):
    ...
```

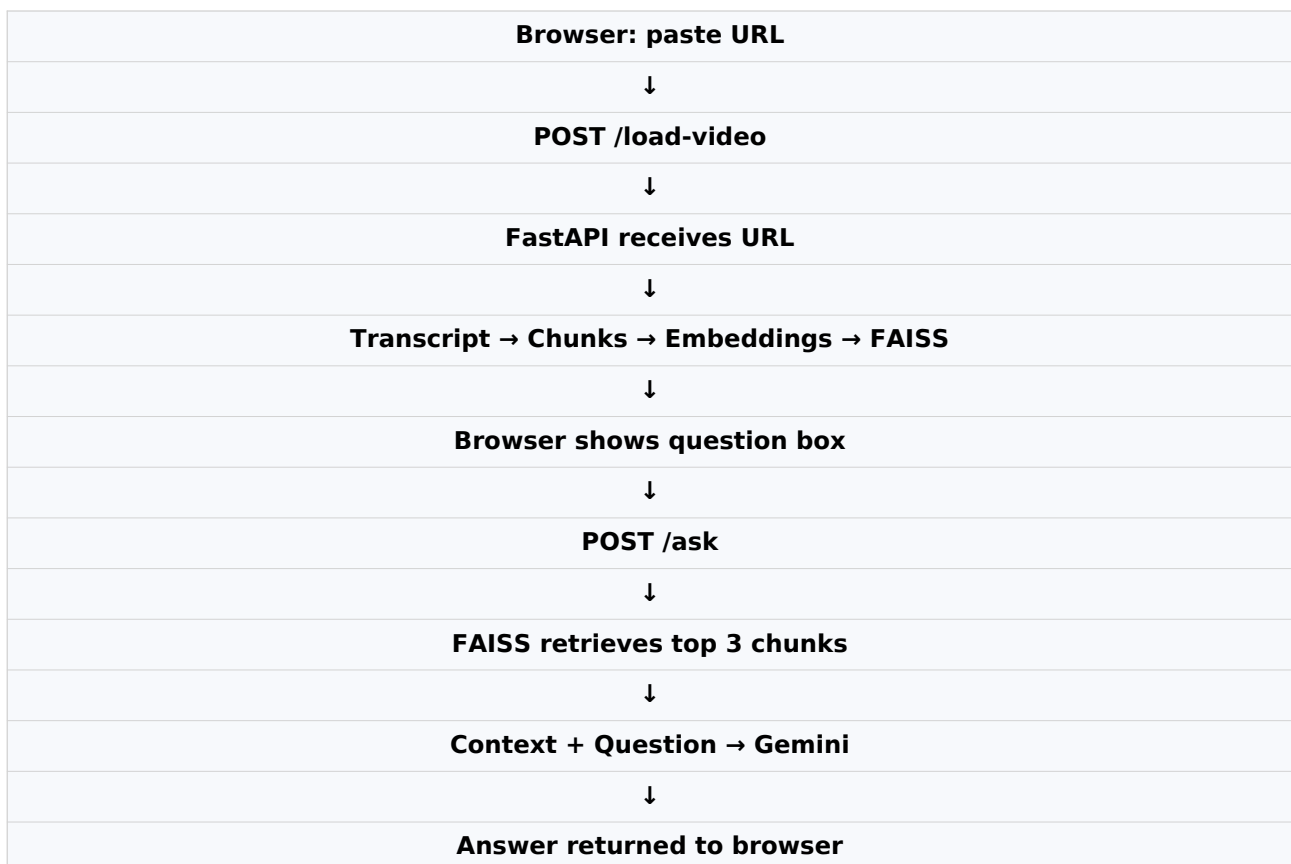
POST /ask receives the question, retrieves relevant chunks, calls Gemini, and returns the answer.

Why vector_store = None?

```
vector_store = None
```

When the application starts, no video has been loaded, so there is no vector store yet. After a video is loaded, this variable points to the FAISS vector store. In this simple local application, loading another video replaces the previous vector store.

7. Browser → FastAPI → RAG



8. Complete Execution — What Happens?

Step	What happens
1	Open <code>http://127.0.0.1:8000</code> .
2	Paste a YouTube URL.
3	Click Load Video.
4	FastAPI receives the URL.
5	The video ID is extracted.
6	The transcript is fetched.
7	Transcript snippets become plain text.
8	Text is split into chunks.
9	Chunks are embedded and indexed in FAISS.
10	Question box is shown.
11	User asks a question.
12	FAISS finds the top 3 relevant chunks.
13	Chunks are added to the prompt.
14	Gemini generates the answer.
15	FastAPI sends the answer back to the browser.

9. Data Transformation

Stage	Data
Input	YouTube URL
Identification	Video ID
Loading	Transcript snippets
Processing	Plain text
Chunking	Text chunks
Embedding	Numerical vectors
Search	FAISS index
Question	User query
Retrieval	Top 3 chunks
Augmentation	Context + question
Generation	Gemini response
Output	Answer in browser

10. Important Differences

Concept	Simple difference
Transcript vs Text	Transcript is a collection of spoken-text snippets; text is the combined string.
Text vs Chunk	Text is the larger content; chunks are smaller pieces.
Chunk vs Embedding	Chunk is readable text; embedding is its numerical representation.
Embedding vs Vector Store	Embedding is a vector; vector store indexes and searches vectors.

Concept	Simple difference
Retrieval vs Generation	Retrieval finds information; generation creates the answer.
RAG vs LLM	RAG supplies relevant external context; the LLM generates language.
FastAPI vs Uvicorn	FastAPI defines the web application; Uvicorn runs it.
Context vs Prompt	Context is retrieved information; prompt contains instructions plus context/question.

11. Current Limitations

- Basic HTML interface.
- Single global vector store; best for a simple local/single-user application.
- Loading another video replaces the previous vector store.
- Basic YouTube URL parsing.
- Requires an available transcript.
- No persistent vector store, authentication, chat history, or conversation memory.
- Advanced error handling is not yet implemented.
- Answers are returned after generation rather than streamed.

12. Future Improvements

Improvement	Benefit
Streaming responses	Makes answers feel faster.
Cache/persist vector stores	Avoids rebuilding embeddings for the same video.
Better URL validation	Supports more YouTube URL formats.
Error handling	Gives clearer messages for failures.
Chat history/memory	Supports follow-up conversation.
Multi-user support	Separates each user's video data.
Timestamps/sources	Helps users locate information in the video.
Better retrieval/reranking	Can improve the relevance of selected chunks.

13. Problems Encountered & Lessons

YouTube Transcript API

Older examples may use `YouTubeTranscriptApi.get_transcript()`. The installed version used in this project works with an API object and `api.fetch(video_id, languages=[...])`.

Embedding quota / 429

Embedding the chunks manually and then embedding them again through `FAISS.from_texts()` creates unnecessary API requests. The current code lets FAISS perform the embedding once.

Gemini model error

The initially attempted Gemini model was unavailable in the user's environment, so the working configuration was changed to `gemini-3.6-flash`.

Gemini response format

The response content can be structured. The current code extracts the actual text with `response.content[0]["text"]`.

14. Why It May Feel Slow

Operation	Main reason
Transcript loading	External YouTube request.
Embedding	External embedding API requests.
FAISS creation	Index is created after embeddings are generated.
Similarity search	Local vector search; generally quick.
Gemini generation	External LLM request and generation time.

Key point: The question-answering delay is usually dominated by the external Gemini request rather than the local FAISS search. Streaming is a useful future improvement.

15. Simple Project Explanation

One line

A YouTube question-answering application that uses RAG to retrieve relevant transcript content and Gemini to generate answers.

About 1 minute

My project is a basic YouTube Chat RAG application. The user provides a YouTube URL, I extract its transcript, convert it into text, and split it into chunks. The chunks are converted into embeddings and indexed in FAISS. When the user asks a question, FAISS retrieves the three most relevant chunks. These chunks are added to the question as context, and Gemini generates the final answer. FastAPI connects the RAG pipeline to a simple browser page.

16. Final Mental Model

INDEXING: Video → Transcript → Chunks → Embeddings → FAISS
↓
RETRIEVAL: Question → Similarity Search → Top 3 Chunks
↓
AUGMENTATION: Top Chunks + Question → Prompt
↓

Remember: Prepare the video so it can be searched → find the right pieces for the question → give those pieces to Gemini → return the answer.

17. Quick Reference

Item	Current value
Project	YouTube Chat RAG
Backend	FastAPI
Server	Uvicorn
Transcript	YouTube Transcript API
Text splitting	RecursiveCharacterTextSplitter
Embedding	gemini-embedding-001
Vector store	FAISS
LLM	gemini-3.6-flash
Retrieval	similarity_search(query, k=3)
Output	Basic HTML page on localhost